PROYECTO CAPSTONE | DOCUMENTO 14

UniPlan

Sistema web de planificación de horarios universitarios

Documento de arquitectura de software

Integrantes

Maximiliano Guzmán · Iván Tobar

Fecha de elaboración

15-10-2026

Kanban y Backlog

Arquitectura de software de UniPlan

Documento 14 | Versión 1.0 | 15-10-2026

UniPlan es una aplicación web cliente-servidor construida como monolito modular Django. Centraliza la planificación semanal, administra información académica, valida reglas de docencia y genera exportaciones. Este documento describe la implementación observada en una auditoría del código y separa los elementos pendientes de la configuración actual.

1. Introducción

La planificación académica requiere coordinar profesores, grupos, sedes, aulas y periodos. UniPlan ofrece interfaces administrativas para construir horarios y un portal docente para consultarlos. La institución usuaria se mantiene anónima en esta descripción.

2. Propósito del documento

Exponer los componentes, dependencias, datos, reglas, controles y decisiones que explican cómo funciona UniPlan; servir de referencia para mantenimiento, pruebas y preparación de un futuro despliegue productivo.

3. Alcance

Se describe el estado del repositorio auditado: configuración Django, aplicaciones, modelos, vistas, formularios, servicios, recursos del navegador, exportaciones, pruebas y ejecución local. Las propuestas de producción se identifican expresamente como pendientes.

4. Contexto del sistema

El navegador solicita páginas y envía acciones por HTTP a Django. Las vistas renderizan plantillas o responden JSON; los servicios y modelos aplican reglas; el ORM persiste en PostgreSQL. Algunas bibliotecas de interfaz se cargan desde CDN. Las descargas Excel y PDF se generan en el servidor como respuestas HTTP.

5. Perfiles de usuario

Perfil	Funciones verificadas
Administrador o planificador	Dashboard, mantenedores, construcción de horarios, validación de conflictos y exportación Excel.
Profesor	Consulta de su horario por periodo, detalle de eventos y exportación PDF; portal de solo lectura.

La autorización se resuelve mediante capacidades y permisos Django. El usuario con capacidades administrativas y perfil docente se dirige primero a la función administrativa; no existe un modelo propio de roles.



Figura 1. Casos de uso principales por perfil, sintetizados desde la auditoría.

6. Requisitos y restricciones arquitectónicas

La solución mantiene reglas de conflicto, disponibilidad, carga y contexto académico en el backend; conserva una sede obligatoria y aula opcional en cada horario; y requiere persistencia relacional para ofertas y grupos. La compatibilidad del archivo con el formato oficial de Academusoft no está acreditada. El proveedor y la configuración productiva siguen pendientes.

7. Estilo arquitectónico

La implementación es un monolito modular Django con patrón MVT y separación parcial de presentación, aplicación, reglas y persistencia. La separación no es estricta: ciertas vistas consultan el ORM y coordinan lógica del negocio, y hay dependencias entre las aplicaciones academico y horarios. El procesamiento ocurre dentro de las solicitudes web.

8. Arquitectura general

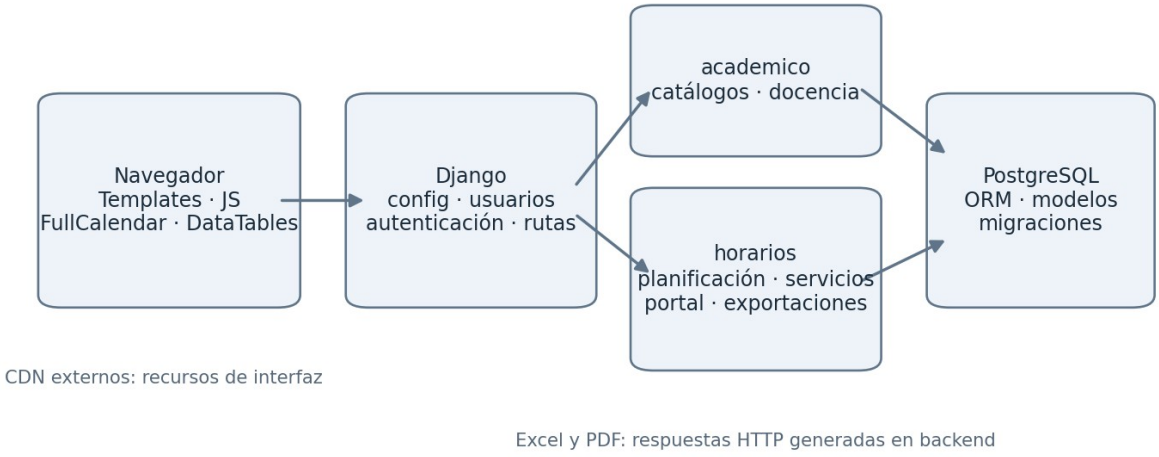


Figura 2. Vista general de componentes de la implementación auditada.

config integra el enrutamiento y la configuración; usuarios gestiona el acceso; academico administra catálogos y docencia; horarios coordina ofertas, planificación, portal, reportes y servicios. horarios.services es un módulo Python dentro del proceso Django.

9. Componentes principales

Módulo	Responsabilidad	Interfaz principal
config	Settings, middleware, rutas y entradas WSGI/ASGI.	URLs y configuración Django.
usuarios	Inicio y cierre de sesión; destino según capacidades; perfil docente.	Sesiones, permisos y Profesor.
academico	Catálogos, sedes, aulas, profesores, disponibilidad, carga y mantenedores.	Modelos, formularios, vistas y ORM.
horarios	Ofertas, grupos, planificación, dashboard, portal y exportaciones.	Vistas, formularios, calendario y modelos.
horarios.services	Validación docente, cálculo y operaciones transaccionales.	Funciones internas invocadas por flujos Django.

10. Capas y responsabilidades

Responsabilidad	Implementación observada
Presentación	Django Templates, Bootstrap, CSS propio, JavaScript, FullCalendar y DataTables.
Aplicación	URLs, vistas, formularios, profesor_views.py y exportacion_views.py.
Reglas	clean() de modelos, formularios, horarios.services y validaciones de calendario.py.
Consultas	dashboard.py, contexto_academico.py, calendario.py y presentacion.py.
Persistencia	Modelos, gestores, ORM, migraciones y PostgreSQL.
Exportaciones	exportacion.py con openpyxl y profesor_pdf.py con ReportLab.

11. Tecnologías utilizadas

Tecnología	Versión auditada	Uso
Python	3.13.3	Backend.
Django	6.1	MVT, ORM, sesiones, permisos y formularios.
PostgreSQL	Versión de servidor no determinada	Persistencia relacional.
psycopg / psycopg-binary	3.3.6	Conexión PostgreSQL.
python-dotenv	1.2.4	Carga de configuración.
Bootstrap	5.3.8	Interfaz.
FullCalendar	6.1.21	Calendario semanal.
DataTables	3.1.3; Buttons 4.1.2	Tablas e interfaz de exportación.

Tecnología	Versión auditada	Uso
openpyxl	3.1.5	Excel.
ReportLab	5.0.1	PDF.

El navegador utiliza JavaScript propio y recursos alojados en jsDelivr y cdn.datatables.net. El repositorio auditado no identifica una SPA independiente ni una API REST separada.

12. Modelo de dominio y persistencia

PostgreSQL almacena 19 modelos de dominio: 15 en academico y cuatro en horarios. Django se conecta mediante el ORM; la auditoría no encontró migraciones pendientes. DB_NAME, DB_USER, DB_PASSWORD, DB_HOST y DB_PORT se obtienen del entorno. El nombre local de la base es uniplan_db.

Área	Modelos principales
Territorio	CentroTutorial, Sede, Aula.
Estructura	Facultad, ProgramaAcademico, ProgramaCentroTutorial.
Currículo	PeriodoAcademico, Semestre, Modalidad, AreaAcademica, Asignatura, PlanEstudioAsignatura.
Docencia	Profesor, DisponibilidadProfesor, CargaDocente.
Planificación	Grupo, OfertaAcademica, OfertaGrupo, Horario.

OfertaGrupo representa una relación explícita entre ofertas y grupos, con cupos por grupo. Profesor.usuario se asocia opcionalmente a User. Horario exige sede y permite aula pendiente. Los conflictos temporales se verifican en la aplicación; no se identificaron restricciones de exclusión temporal en PostgreSQL.



Figura 3. Relaciones centrales del dominio; vista parcial de los 19 modelos.

13. Procesamiento de la planificación

El planificador selecciona un contexto académico y consulta la proyección semanal. Las acciones del calendario envían datos al backend, que resuelve la oferta, comprueba pertenencia al contexto, ejecuta validaciones y guarda mediante servicios. El resultado se devuelve a la interfaz. El calendario proyecta día y horas sobre una semana de referencia: los registros son patrones semanales, no eventos con fecha individual.

14. Reglas de negocio

Regla	Comportamiento implementado
Conflictos	Se rechazan superposiciones del profesor, grupos compartidos y aula física en periodo y día; profesor entre sedes.
Tiempo y lugar	Fin posterior al inicio; duración mínima de una hora; aula, si existe, perteneciente a la sede.
Docencia	Disponibilidad activa cubre completamente el intervalo; carga activa limita las horas semanales.
Transversalidad	Una oferta sirve varios grupos sin duplicar el cómputo docente del horario.
Contexto y edición	Se valida pertenencia académica; mover conserva duración y redimensionar modifica el fin.

La disponibilidad y la carga se calculan por profesor y periodo, sin limitarse a los filtros visibles. La validación docente se omite para horario u oferta inactivos. No se verificó una comparación automática entre capacidad de aula y cupos.

15. Seguridad, autenticación y autorización

Django utiliza sesiones, LoginView y LogoutView, permisos por modelo y operación, middleware y protección CSRF. Formularios y endpoints aplican validación del lado servidor. Los selectores auxiliares de catálogos no presentan todos el mismo nivel de permiso; algunos exigen únicamente inicio de sesión.

El Portal Profesor obtiene la identidad desde request.user y filtra eventos, detalle y PDF por el profesor asociado. Su FullCalendar se configura sin edición. La configuración actual conserva parámetros de desarrollo; la configuración segura para producción está pendiente.

16. Transacciones, concurrencia y consistencia

operacion_planificacion() abre transaction.atomic(), identifica profesores afectados y los bloquea por clave primaria ascendente con select_for_update(). Los servicios bloquean recursos adicionales según la operación, ejecutan full_clean() y validaciones, guardan y revalidan cuando corresponde. Una excepción que sale de la transacción revierte la operación.

Operación	Bloqueos explícitos relevantes
Guardar horario	Profesor(es), oferta, horario existente, aula seleccionada y grupos asociados.
Guardar oferta	Profesor anterior y nuevo, oferta, aulas y grupos relacionados.
Guardar OfertaGrupo	Profesor(es), oferta, aulas y grupos actuales o adicionales.
Eliminar planificación	Profesor(es) identificados por servicio o QuerySet protegido.

El protocolo protege flujos concretos, no todas las escrituras del sistema. Horario y OfertaAcademica restringen ciertas operaciones masivas mediante PlanificacionQuerySet. La gestión de disponibilidad y carga no aplica por sí misma una revalidación global de todos los horarios existentes.

17. Interfaz web y comunicación cliente-servidor

Las páginas renderizadas por Django incorporan Bootstrap y CSS propio. JavaScript usa fetch y respuestas JSON para filtros, calendario y acciones concretas. FullCalendar permite crear por arrastre, mover y redimensionar bloques semanales; DataTables permite explorar tablas. El backend resuelve la autorización y las reglas definitivas.

18. Exportaciones e integraciones

Elemento	Estado y comportamiento
Excel	Implementado con openpyxl; /planificacion/exportar/ y /planificacion/exportar-excel/. Archivo tabular de 16 columnas, con una fila por horario y grupo asociado; se entrega por HTTP y protege texto frente a fórmulas.
PDF docente	Implementado con ReportLab; /profesor/horario/exportar-pdf/. Genera en respuesta HTTP el horario autorizado del profesor autenticado.
PostgreSQL y CDN	Conexión ORM/psycopg implementada; bibliotecas de interfaz referenciadas desde CDN.
Academusoft	Exportación tabular propia implementada. No se identificó API directa ni validación acreditada contra un formato institucional oficial.

19. Despliegue actual y propuesta productiva

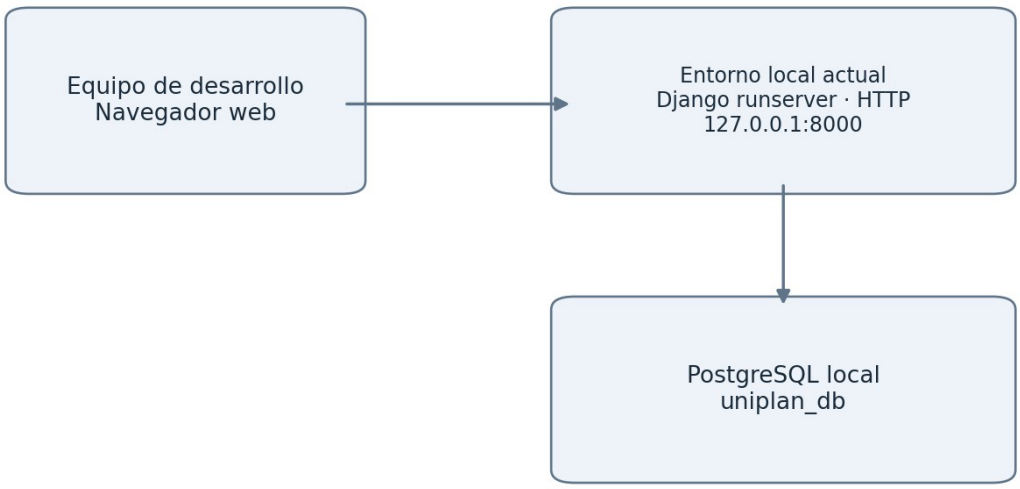


Figura 4. Despliegue local observado y aspectos productivos pendientes.

Aspecto	Actual	Pendiente
Servicio	Django runserver por HTTP local.	Servidor productivo y HTTPS.
Base	PostgreSQL local uniplan_db.	Operación y respaldos productivos.
Estáticos	staticfiles y CDN externos.	Publicación de estáticos.
Automatización	No se identificó pipeline de despliegue.	Definir proveedor y procedimiento.

WSGI y ASGI existen como puntos de entrada en el proyecto; su existencia no acredita un despliegue productivo. No se encontró configuración de proveedor de nube, servidor productivo ni HTTPS.

20. Decisiones arquitectónicas

Los motivos de la siguiente tabla son inferencias técnicas a partir de la implementación; no se presentan como decisiones documentadas formalmente por el equipo.

Decisión observable	Motivo inferido	Impacto
Monolito Django	Simplificar integración.	Un proceso con módulos relacionados.
PostgreSQL y ORM	Conservar relaciones y bloqueos.	Persistencia y transacciones centralizadas.
Templates y JavaScript	Combinar páginas y acciones dinámicas.	Sin SPA separada.
FullCalendar	Editar el patrón semanal.	Traducción entre calendario y horarios.
Servicios y validación backend	Compartir reglas.	Control definitivo en servidor.
Sede obligatoria; aula opcional	Permitir asignación posterior.	Ubicación conocida con aula pendiente.
OfertaGrupo explícito	Representar transversalidad.	Grupos asociados sin duplicar sesión.
Exportación backend	Generar con datos autorizados.	Descargas Excel y PDF en servidor.

21. Calidad y pruebas

Comprobación	Resultado de la auditoría
Django	415 pruebas descubiertas: academico 52, horarios 356, usuarios 7. La suite no fue ejecutada; no se afirma que pase.
JavaScript	108 pruebas ejecutadas y aprobadas en cinco archivos con simulación de DOM; no equivalen a pruebas completas de navegador.
Configuración	manage.py check ejecutado sin incidencias.
Cobertura y carga	No se midieron porcentaje de cobertura ni tiempos bajo carga.

22. Restricciones y limitaciones

La configuración auditada es de desarrollo. No se acreditó el formato oficial de Academusoft ni una comparación automática entre capacidad de Aula y cupos. El modelo semanal no cubre feriados o excepciones fechadas; la separación de capas es parcial, la protección concurrente se limita a flujos concretos y no hay mediciones bajo carga.

23. Aspectos pendientes

Tema	Trabajo pendiente
Producción	Definir infraestructura, HTTPS, variables, secretos, estáticos, respaldos y automatización.
Intercambio institucional	Obtener plantilla o especificación y validar la compatibilidad del Excel Academusoft.
Calidad	Ejecutar la suite Django y registrar resultados; medir cobertura y rendimiento si se requieren.
Evolución funcional	Evaluar aforo de aulas y manejo de excepciones fechadas según requisitos validados.

24. Conclusión

La arquitectura actual permite centralizar la planificación académica, separar capacidades administrativas y docentes, validar reglas en el backend y persistir horarios en PostgreSQL. Los servicios protegen determinadas operaciones concurrentes y las exportaciones Excel y PDF están implementadas. El despliegue productivo y la validación institucional del formato de intercambio siguen pendientes.

25. Anexos y diagramas

Las cuatro figuras son vistas de síntesis elaboradas desde la auditoría. La vista de clases es deliberadamente parcial: la tabla de la sección 12 enumera el dominio completo. Las figuras no atribuyen infraestructura productiva a la implementación actual.

Figura prevista	Ubicación o finalidad
Casos de uso	Figura 1, perfiles y funciones principales.
Componentes	Figura 2, arquitectura general del monolito.
Clases	Figura 3, relaciones principales; vista parcial de los 19 modelos.
Despliegue	Figura 4, entorno local observado y producción pendiente.

Fuente técnica

Auditoría técnica del repositorio UniPlan entregada por el equipo el 15-10-2026. Sus hallazgos se utilizan como fuente del estado implementado. Las razones de decisiones presentadas en la sección 20 se identifican expresamente como inferencias.